

1. Tipo de Aplicación

La aplicación se desarrolla como una aplicación Android nativa utilizando Kotlin y Jetpack Compose.

Factor	Justificación
Rendimiento	Las aplicaciones nativas acceden directamente a las APIs del sistema operativo Android, garantizando una renderización fluida y tiempos de respuesta óptimos sin capas extras
Consistencia visual	Con Jetpack Compose se puede implementar Material Design 3 de forma nativa, asegurando coherencia con las guías de diseño de Android.
Plataforma objetivo	El sistema está dirigido exclusivamente a usuarios de Android. Por lo que no se pide algo más extra y no se requiere.
Ecosistema tecnológico	Android Studio, las librerías Jetpack, Navigation Compose y las herramientas de prueba conforman un ecosistema robusto, bien documentado lo que permite su implementación.

Un enfoque híbrido introduciría complejidad extra y no se justifica para este sistema de plataforma única.

2. Patrón de Arquitectura – MVVM

La aplicación sigue el patrón MVVM (Model – View – ViewModel), recomendado oficialmente por Google como la arquitectura estándar para el desarrollo Android moderno con Jetpack Compose.

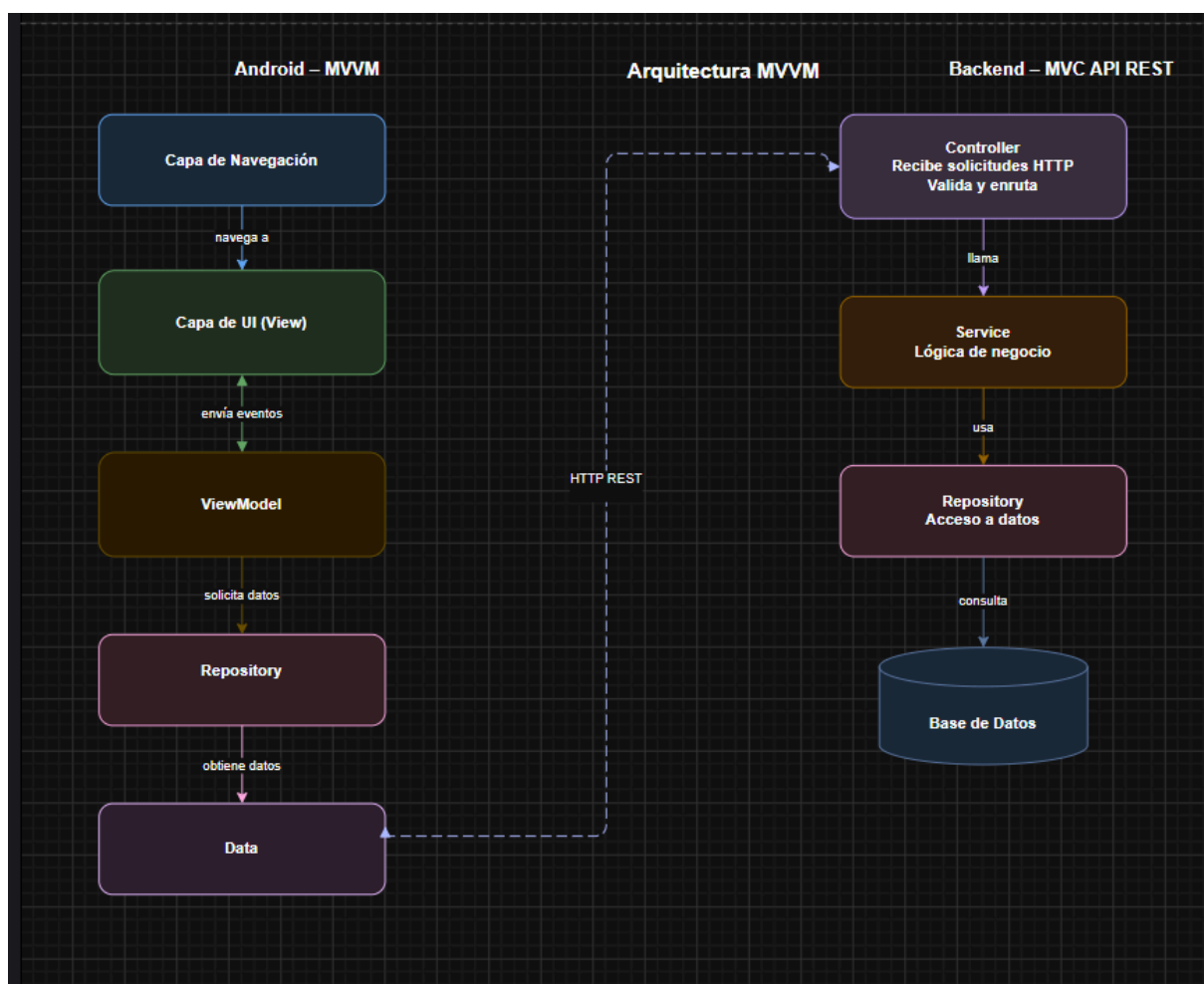
Descripción de las capas

- La capa de vista es la encargada de presentar la información al usuario y obtener sus interacciones. Está compuesta por muchas pantallas y componentes visuales que reaccionan automáticamente a los cambios de estado que provienen de la capa inferior.
- La capa ViewModel actúa como mediador entre la vista y los datos. Esta se encarga del estado de la interfaz y procesa los eventos generados por el usuario, dando el resultado hacia la vista de forma reactiva.
- La capa Repositorio abstrae el origen de los datos. Su responsabilidad es proveer la información que solicita el ViewModel sin que este necesite conocer de dónde proviene, ya sea de una fuente local o remota.
- La capa de Fuente de Datos es quien obtiene los datos directamente al backend.

¿Por qué MVVM?

- Separación de las responsabilidades: los Composables de UI no contienen lógica de negocio.
- Resistencia a cambios de configuración: los ViewModels sobreviven a rotaciones de pantalla.
- Testabilidad: cada capa puede probarse de forma aislada.
- Escalabilidad: conectar un backend real solo requiere implementar
- RemoteDataSource con Retrofit. La UI y el ViewModel no necesitan modificaciones.
- Integración nativa con Jetpack lo que permite un mejor acoplamiento

3. Diagrama de Arquitectura



4. Flujo General del Sistema

A continuación se describe el flujo de extremo a extremo de la aplicación:

- Al iniciar la aplicación, se carga la pantalla principal donde se muestra el listado de espacios de coworking disponibles. Cada espacio se presenta como una tarjeta con su información básica.
- Cuando el usuario selecciona un espacio, la aplicación navega a una pantalla de detalle donde se muestra la información completa del espacio seleccionado, junto a la opción de realizar una reserva.
- Al confirmar la reserva, el sistema procesa la solicitud y notifica al usuario el resultado. En la versión actual se simula esta confirmación, mientras que en la versión final la solicitud se enviará al servidor para ser registrada.
- La aplicación cuenta con una barra de navegación inferior que permite al usuario alternar entre el listado de espacios disponibles y el historial de sus reservaciones en cualquier momento.